1. Introduction

1.1 Motivation

Railway transportation remains relevant even after its long history due to its efficiency, safety, and sustainability. However, the most limiting factor has always been the use of the required infrastructure. As such, the critical problem of utilizing the railway effectively has always been scheduling. The complexity of this problem is continuously increasing with the size and intricacy of the railway system as well as the demand to use it.

Traditionally, scheduling was done manually by humans, only with the advancements of optimizing algorithms and computational power have humans been aided by computers. This still produces a static schedule that might be unusable if a disruption occurs. This is where train dispatching comes to play where the decisions are done in real-time, managing delays and conflicts as they happen. Real-time dispatching is often still only done by humans and provides a challenge for optimization algorithms as they need to find an acceptable solution in a matter of minutes or even seconds.

1.2 Goals

The main goal of this thesis is to develop a heuristic that is capable of train dispatching in real-time with interpretable results - a human should be able to understand the structure of the result to verify its validity.

The heuristic will be evaluated on the *DISPLIB*: Train dispatching benchmark library (Kloster et al., 2025b) consisting of train dispatching problem (TDP) instances from real world data. This library of instances was introduced for an optimization competition.

A significant part of this thesis is focused on the aggregation of the problem as presented in the *DISPLIB*. This is the main innovation which allows us to use a top-down approach, reducing the problem into selecting the routes top-down and then scheduling it with a more concise model, as opposed to the bottom-up approach, building up the solutions by sequentially routing and resolving conflicts, which is used by the winning solutions in the *DISPLIB* competition.

1.3 Thesis structure

This thesis is structured in the following chapters with their contents:

1. *Introduction* - outlines the motivation, goals and structure of this thesis.
2. *Literature review* - review of the pertinent literature separated into three parts, literature connected to the *DISPLIB*, the most relevant resources inspiring the heuristic or providing key theoretical ideas, and an overview of selected state-of-the-art in scheduling optimization.
3. *Theoretical background* - basic theory necessary for the description of the solutions. The main focus is on graph topology and the use of alternative and disjunctive graphs for scheduling models.
4. *Methodology* - detailed description of the problem, aggregation, routing and conflict resolution.
5. *Results* - presentation of performance of the algorithms described in methodology on the *DISPLIB* instances.
6. *Conclusion* - discussion of results and highlight of main ideas presented in this thesis.

2. Literature review

This chapter provides a review of the relevant literature separated into three parts. The first is literature relevant to the *DISPLIB*. The second are the most relevant resources inspiring the heuristic or providing key theoretical ideas. Third is an overview of selected state of the art in scheduling optimization.

2.1 DISPLIB

As the goals of this thesis will be evaluated on the instances in the *DISPLIB* library, we provide a review of the introductory paper as well as a description of the competition winning solutions.

2.1.1 Problem and library

The train dispatching problem (TDP) which this thesis is working with is presented in *DISPLIB: A library of train dispatching problems* (Kloster et al., 2025b) alongside the collection of instances which can be used to benchmark this problem. This provides an important step for the subfield of train dispatching optimization - a unified problem format and real-life data to be used for benchmarking. The instances and competition results can be accessed from `https://displib.github.io/`.

> ... each study (on train scheduling) is typically tightly connected with a specific industrial use case. Code and data are seldom shared publicly. This fact hinders reproducibility, and has led to a proliferation of papers describing algorithms for more or less compatible problem definitions, without any real opportunity for readers to assess their relative performance. Inspired by the successful communities around MILP, SAT, TSP, VRP, etc., we introduce a common problem definition and file format, *DISPLIB*, which captures all the main features of train re-routing and re-scheduling. (Kloster et al., 2025b)

The instances are published with best known solutions (assumed with unlimited time), for the 10-minute computation time limit only the competition team rankings are presented, without overview of the actual solutions.

2.1.2 Competition and solutions

Part of the presentation of *DISPLIB* was also the announcement of the competition for optimization using this library. The competition concluded at the Optimization and Decision

Science Conference (ODS) 2025 where they presented the winners. Unfortunately the materials for the solutions are not available, and only one team wrote a paper about their solution. Still, we find it essential to provide the reader with the description for other solutions. Consequently, the limited information from the book of abstracts for the ODS 2025 (Barbato et al., 2025) is summarized.

First place went to Carolin Scholl and Luka Stärk (DB Systel GmbH). Their solution is based on branch and bound inspired by Monte Carlo Tree Search (D'Ariano et al., 2007), prioritizing not only lower bound improvement but also search for feasible solutions. In each step of the tree they resolve the chronologically first conflict either by setting precedence to either of the trains using the conflicting resource or by rerouting either train if possible. This is a bottom-up approach, only adding the necessary constraints for current conflicts.

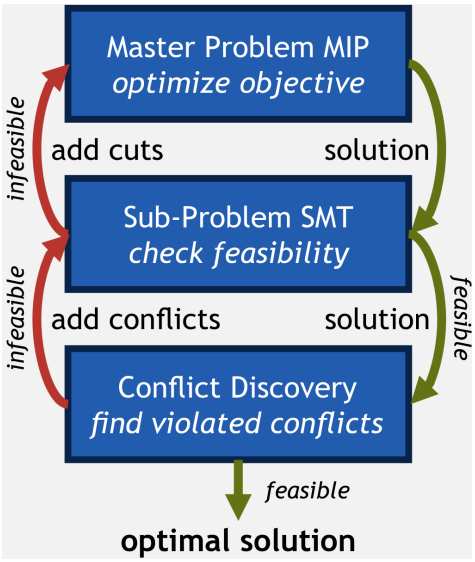
Second place went to the Openbus team (ETH Zürich) which published a description of their solution in *Handling Conflicts in Microscopic Railway Timetabling at Scale: A Hybrid Approach*[1] (Dubach et al., 2025b). Their solution utilizes 2 distinct solvers which run in parallel. First one is a Logic-Based Benders Decomposition (LBBD) procedure (Leutwiler & Corman, 2022) where a master mixed-integer programming (MIP) problem performs a selection of current available paths, a slave satisfiability modulo theory (SMT) solver creates a feasible timetable if possible (without conflicts) and a subproblem which detects conflicts and adds them to the SMT solver, which in turn adds cuts to the master problem to maintain feasibility. Figure 2.1 shows the diagram of the solver as presented in their poster. Second solver uses a simpler MIP representation which iteratively relaxes the bounds for conflicting resources and tightens the bounds for nonconflicting resources converging to a feasible solution. Both of these solvers use a bottom-up approach, after a bound tightening procedure is applied globally. The authors argue that both of these solvers exploit different instance structures and share their bounds to improve convergence.

Third place went to Venislav Steliyanov Varbanov (independent). This solution also uses an MIP model with a decomposition to reduce the number of time variables - this idea is also used by the heuristic in this thesis, inspired by the provided description. After this aggregation of time variables, the conflicts are again resolved sequentially, similarly to the other 2 solutions.

We can see that while the winning solutions may differ in their model representation and optimization approach, and 2 of them use a certain level of aggregation, they have one thing in common - they resolve the conflicts sequentially, and thus they only need to consider the conflicts that arise in the current partial solution and therefore the number of constraints and variables necessary to prevent the conflicts stays limited.

[1] This paper is not officially published, only a pre-print was available.

Figure 2.1
Openbus DISPLIB competition poster



Source: ETH Zürich Research Collection (Dubach et al., 2025a)

2.2 Theory and heuristics

While the *DISPLIB* winning solutions provided us with the ideas of aggregation of variables and combining conflict resolutions, they provided very little on the actual implementation. This section describes the relevant sources to explain the necessary theory to model the problem effectively.

Referenced by the winning solution is *A Branch and Bound Algorithm for Scheduling Trains in a Railway Network* (D'Ariano et al., 2007). It uses an alternative graph formulation for the train scheduling problem. With this representation we can model resource conflicts with alternative precedence arcs. It also describes the idea of inferred selection, meaning that by choosing one precedence alternative for a resource implies selection on other resources (the alternative is forbidden as it would produce an infeasible solution).

Job-shop Scheduling with Blocking and No-wait Constraints (Mascis & Pacciarelli, 2002) introduces the alternative graph concept as a generalization of the disjunctive graph (Roy & Sussmann, 1964). The alternative graph is used to formulate a model for the blocking job-shop problem (BJSP) that is closely related to TDP.

Improving Real-Time Train Dispatching: Models, Algorithms and Applications. (D'Ariano, 2008, p. 84-85) provides static implication rules for the alternative graph representation that can be used for inferred selection based on the structure of TDP.

Railway scheduling problems and their decomposition (Strotmann, 2007) is a thesis that shows decomposition techniques for train scheduling. It inspired us to focus on the event graph used in this thesis and to separate the routing and conflict resolution problems.

2.3 State of the art

Because the description of the solutions that are directly connected to this thesis is limited, we provide the reader with an overview of the recent state-of-the-art solutions for closely related problems in train dispatching and scheduling.

Optimizing train dispatching for the Union Pacific Railroad (Boccia et al., 2025) presents an automated dispatching system that is in use on a real rail network. It uses a decomposition of the problem and global synchronizer to resolve conflict. It is a prime example how optimization can improve real-life solutions.

A Branch-and-Cut Algorithm for Scheduling Train Platoons in Urban Rail Networks (Chai et al., 2024) describes a mixed-integer linear problem (MILP) for scheduling of train platoons, a problem where different vehicles are flexibly and dynamically coupled and decoupled.

Authors in *Decentralized Multi-Agent Coordination for Real-Time Railway Traffic Management* (D'Amato et al., 2025) propose the use of another emerging idea - instead of central or area based decomposition schedulers, trains cooperate as separate agents to prevent conflicts and achieve efficient scheduling. This decentralized nature of cooperation has already been used on drone related problems with success (Hert et al., 2023).

3. Theoretical background

This chapter will cover the necessary theory used to formulate and solve our problem. Main focus will be on graph topology and alternative graphs and their relation to the train dispatching problem.

3.1 Basic definitions

While some of these definitions may feel redundant, we will use them as an opportunity to establish the notation. Most of the definitions are paraphrased from *Handbook of Graph Theory* (Gross et al., 2013) with notation adjusted to ours.

Definition 3.1. Graph is an ordered pair $G = (V, E)$, where $V = \{v_1, \ldots, v_{|V|}\}$ is a set of vertices[1] and $E = \{e_1, \ldots, e_{|E|}\}$ is a set of edges, where edge $e = \{u, v\}$, $u, v \in V$ is an *unordered* pair of vertices.

Definition 3.2. Directed graph (digraph) is an ordered pair $G = (V, E)$, where $V = \{v_1, \ldots, v_{|V|}\}$ is a set of vertices and $E = \{e_1, \ldots, e_{|E|}\}$ is a set of edges[2], where edge $e = (u, v)$, $u, v \in V$ is an *ordered* pair of vertices.

Definition 3.3. Multigraph is an ordered tuple $G = (V, E, r)$, where $V = \{v_1, \ldots, v_{|V|}\}$ is a set of vertices, $E = \{e_1, \ldots, e_{|E|}\}$ is a set of edges and $r : E \to \{\{u, v\} \mid u, v \in V\}$ is a mapping from edges to *unordered* pairs of endpoint vertices.

Definition 3.4. Directed multigraph is an ordered tuple $G = (V, E, s, t)$, where $V = \{v_1, \ldots, v_{|V|}\}$ is a set of vertices, $E = \{e_1, \ldots, e_{|E|}\}$ is a set of edges and $s : E \to V$ is a source (start) mapping from edges to vertices and $t : E \to V$ is a target (end) mapping from edges to vertices.

Example of each type of graph is in figure 3.1.

Remark. As you can see in figure 3.1, digraph allows for parallel edges (2, 3) and (3, 2). This will be a useful property for the alternative graph definition.

Since we will mostly be working with directed graph, the following definitions are tailored for directed graphs. The following definitions of operators also abuse the notation by omitting the specification of graph G, which will be included in the context of multiple graphs if necessary.

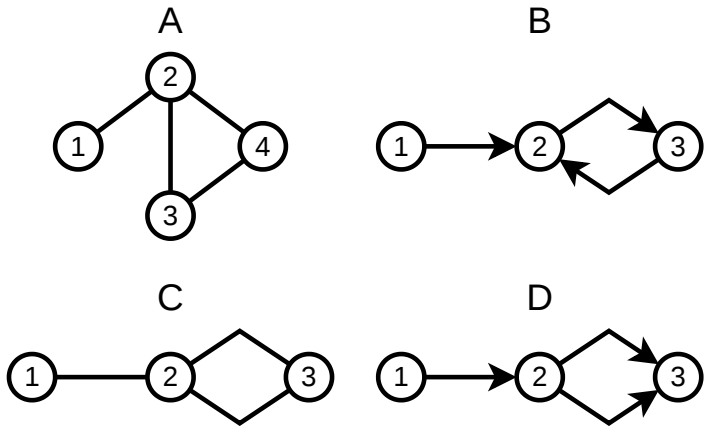
Definition 3.5. Source (start) $\mathrm{s}(e) = u$, $e = (u, v)$ is the starting vertex of edge e.

Definition 3.6. Target (end) $\mathrm{t}(e) = v$, $e = (u, v)$ is the ending vertex of edge e.

[1] Vertices are often called nodes.

[2] Commonly the edges in directed graphs are called arcs.

Figure 3.1
Graph types



Note: (A) is graph, (B) is directed graph, (C) is multigraph, and (D) is directed multigraph.

Definition 3.7. Incoming edges $\delta_{\text{in}}(v) = \{e \mid t(e) = v,\, e \in E\}$ of vertex v is a set of edges e that end in v.

Definition 3.8. Outgoing edges $\delta_{\text{out}}(v) = \{e \mid s(e) = v,\, e \in E\}$ of vertex v is a set of edges e that start in v.

Definition 3.9. Predecessors (In-neighborhood) $\mathrm{N}_{\text{in}}(v) = \{u \mid u = \mathrm{s}(e),\, e \in \delta_{\text{in}}(v)\}$ of vertex v is a set of vertices u that are connected to v via its incoming edges.

Definition 3.10. Successors (Out-neighborhood) $\mathrm{N}_{\text{out}}(v) = \{w \mid w = \mathrm{t}(e),\, e \in \delta_{\text{out}}(v)\}$ of vertex v is a set of vertices w that are connected to v via its outgoing edges.

Definition 3.11. In-degree $\mathrm{d}_{\text{in}}(v) = |\,\delta_{\text{in}}(v)|$ is the number of incoming edges of v.

Definition 3.12. Out-degree $\mathrm{d}_{\text{out}}(v) = |\,\delta_{\text{out}}(v)|$ is the number of outgoing edges of v.

Definition 3.13. Path is $P = \mathrm{P}(u,\, v)$ from u to v is a sequence of edges $P = (e_1,\, \ldots,\, e_{|P|})$, such that $\mathrm{t}(e_{i-1}) = \mathrm{s}(e_i)$, $2 \leq i \leq |P|$, $u = \mathrm{s}(e_1)$ and $v = \mathrm{t}(e_{|P|})$.

Definition 3.14. Cycle is a path that starts and ends in the same vertex.

3.2 Graph topology

Because event graphs described later in section 3.3.1 require certain properties or concepts such as acyclicity and topological ordering (Bang-Jensen & Gutin, 2008), this section will define and explain them.

Definition 3.15. Directed acyclic graph (DAG) is directed graph that contains no cycles.

Definition 3.16. Topological order O is a sequence of vertices in a directed graph $O = (v_1, \ldots, v_{|V|})$ such that $e = (v_j, v_j) \implies v_i \prec v_j$ ($i < j$, v_i appears before v_j in the sequence).

Theorem 3.1. Topological order can only be created for directed acyclic graphs.

Proof. By contradiction, for path starting at u and ending at v, $u \prec v$ in the topological order, because for each $e = (x, y)$ in path, $x \prec y$ and ordering $\prec$ is transitive. For a cycle, u and v are in the same spot in the order, this is a contradiction. □

To detect a cycle and get a topological sort we use algorithm 3.1, it is a combination of depth first search (DFS) for cycle detection and DFS for topological sorting (Cormen et al., 2022). It runs in $\mathrm{O}(|V| + |E|)$ time complexity with $\mathrm{O}(|V|)$ required space. It allows us to do both cycle detection and topological sort in one pass of DFS.

Remark. Algorithm 3.1 uses binary operator + for concatenation of sequences, so for $S_1 = (a, b)$, $S_2 = (c, d)$, $S_1 + S_2 = (a, b, c, d)$.

Algorithm 3.1

Topological sort with cycle detection

```
input: directed graph G(V, E)
output: () if G contains a cycle else topological order O

function DFS(v, S)
  if label(v) = InRecursion
    return True
  else if label(v) = Done
    return False
  end

  label(v) ← InRecursion

  foreach w ∈ N_out(v)
    if DFS(w, S)
      return True
    end
  end

  S ← S + (v)
  label(v) ← Done

  return False
end

begin
  foreach v ∈ V(G)
    label(v) ← NotDone
  end

  S ← ()

  foreach v ∈ V(G)
    if DFS(v, S)
      return ()
    end
  end

  O ← reverse(S)
  return O
end
```

3.3 Alternative graphs

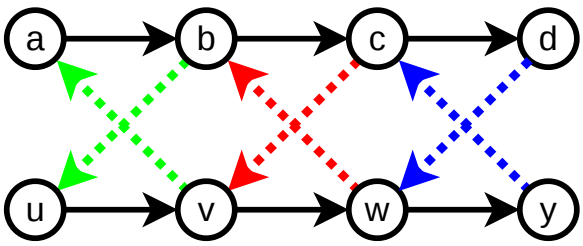
Alternative graphs were introduced in (Mascis & Pacciarelli, 2002). They provide a very useful framework for modeling resource allocation problems with blocking. They are a generalization of disjunctive graphs (Roy & Sussmann, 1964).

Definition 3.17. Alternative graph is an ordered tuple $G = (V, E, A)$, where V are vertices and E are edges, same as for directed graph, and $A = \{a_1, \ldots, a_{|A|}\}$ is a set of disjunctive (alternative) edge pairs $a = ((u, v), (x, y))$, $u, v, x, y \in V$.

Remark. We denote the alternative $\bar{e}$ to the edge e from the disjunctive pair $a = (e, \bar{e})$.

To illustrate the concept of alternative graphs, figure 3.2 shows an alterative graph with 3 disjunctive pairs.

Figure 3.2
Alternative graph



Note: Alternative edges are dotted, alternative pairs grouped by color.

Definition 3.18. A selection S is a set of edges obtained from A by choosing at most one edge from each pair. The edges in this selection are selected, their alternatives (the other edge from the pair) is forbidden.

Definition 3.19. Selection S is complete if exactly one edge is chosen from each pair, otherwise it is a partial selection.

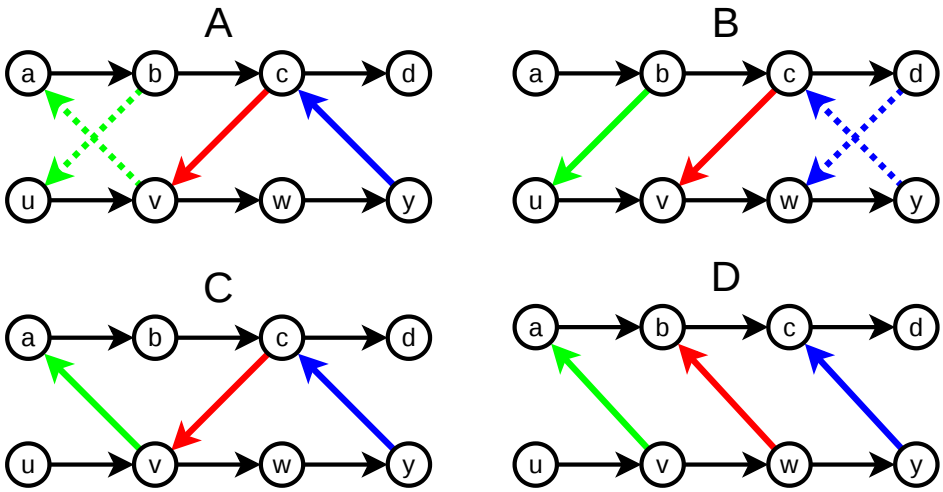
Definition 3.20. Selection S is consistent if graph $(V, E \cup S)$ does not contain a cycle. If the graph does contain a cycle, the selection is inconsistent.

To illustrate the selection terms, figure 3.3 shows different types of selections from on alternative graph from figure 3.2.

Definition 3.21. Alternative edge e is inferred from selection S, $S \vdash e$ if $S \cup \{\bar{e}\}$ is an inconsistent selection, meaning that the choice of e from alternative pair $(e, \bar{e})$ can be inferred from S because choosing $\bar{e}$ would be inconsistent. Similarly, we can say that one alternative edge x implies another y, $x \implies y$ if $\{x\} \vdash y$.

An example from the graph in figure 3.2, we can see that $(b, u) \implies (c, v)$ and $(v, a) \implies (w, b)$ etc.

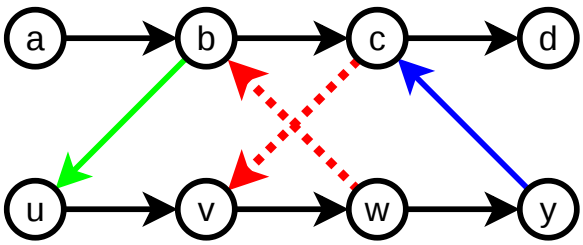
Figure 3.3
Alternative graph with selections



Note: (A) is partial inconsistent, (B) is partial consistent, (C) is complete inconsistent, and (D) is complete consistent selection. Alternative edges are dotted, selected edges are full, alternative pairs grouped by color.

Remark. It is possible to have a partial consistent selection P for which there exist no complete consistent selection C, $P \subset C$. One such case is in figure 3.4, where $(b, u) \implies (c, v)$ and $(y, c) \implies (w, b)$. Because (c, v) and (w, b) are in the same disjunctive pair, we cannot make a consistent selection for this pair.

Figure 3.4
Partial consistent selection without possible complete consistent selection



Note: Alternative edges are dotted, selected edges are full, alternative pairs grouped by color.

3.3.1 Weighted graphs

To extend the descriptive ability of graphs to model problems we assign properties to vertices or edges. One of the most common ones is to add weights to edges.

Definition 3.22. Weighted directed graph is an ordered tuple $G = (V, E, w)$, where V and E are same as in digraph definition 3.2, and $w : E \to \mathbb{R}$ is an edge weight mapping.

Definition 3.23. Length $\mathrm{L}(\cdot)$ of path P is the sum of the weights of each edge,

$$\mathrm{L}(P) = \sum_{e \in P} w_e \tag{3.1}$$

Definition 3.24. Length of the longest path $\mathrm{L}_{\max}(\cdot)$ from u to v is

$$\mathrm{L}_{\max}(u, v) = \max_{P \in \mathcal{P}_{u,v}} \mathrm{L}(P), \tag{3.2}$$

where $\mathcal{P}_{u,v}$ is the set of all paths from u to v.

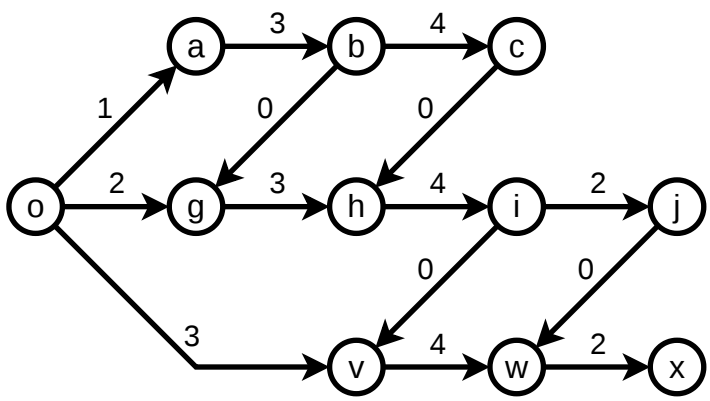
Remark. The length of the longest path can only be calculated for a graph without any positive length cycles, otherwise we could simply go in the cycle indefinitely to prolong the path (Gross et al., 2013).

Event graphs

In our model, we will use a weighted DAG that we call an *event graph*, where each vertex corresponds to an event, edge direction assigns precedence of events and the weight of an edge will be corresponding to a minimal duration between events. All events share the origin o which determines the start of the timeline, and it is used to calculate the time of each following event.

Example Figure 3.5 shows an event graph for 3 trains sharing resources. Edges (o, a), (o, g), and (o, u) set the earliest start of the train, horizontal edges define the fastest passing of a section and diagonal edges ensure precedence for resources e.g. edge (b, g) ensures that g starts after b, since section (a, b) required the same resource as (g, h).

Figure 3.5
Event graph for three trains with shared resources



The time of the event x in the event graph can be calculated as the longest path from the origin o to x, $\mathrm{T}(x) = \mathrm{L}_{\max}(o, x)$.

Finding all the paths from o to each $x \in V$ would be very expensive. Using DFS the complexity is exponential (Homer & Selman, 2011).

First optimization is the use of Bellman principle (Light, 2024)(Bellman, 1952),

$$\mathrm{T}(v) = \mathrm{L}_{\max}(o,\, v) = \max_{u \in \mathrm{N}_{\mathrm{in}}(v)} \mathrm{L}_{\max}(o,\, u) + w_{(u,\, v)}. \tag{3.3}$$

This principle is used by the Floyd-Warshall algorithm (Floyd, 1962) and Dijkstra's algorithm (Dijkstra, 1959) which are used for finding the shortest paths for all pairs of vertices and from a select starting vertex respectively. Both of them could be adjusted to find the longest path within polynomial complexity, $\mathrm{O}(|V|^3)$ for Floyd-Warshall and $\mathrm{O}((|V| + |E|)log|V|)$ for Dijkstra with binary heap.

Since we already use algorithm 3.1 to detect a cycle, we can just process the time of the events in the topological order, because topological order guarantees that any predecessors are already processed.

Definition 3.25. Critical path $\mathrm{CP}(\cdot)$ for vertex v is the longest path from origin o to v.

Critical path limits the earliest start of event v, delaying any event along this path will also delay the event v,

$$\mathrm{L}(\mathrm{CP}(v)) = \mathrm{T}(v). \tag{3.4}$$

Definition 3.26. Critical path edge $\mathrm{CPE}(\cdot)$ of v is the last edge of the critical path for v.

We can again use the Bellman principle to reconstruct the critical path,

$$\mathrm{CP}(v) = \mathrm{CP}(u) \cup \{(u,\, v)\},\ \mathrm{CPE}(v) = (u,\, v). \tag{3.5}$$

Algorithm 3.2 outputs the time $\mathrm{T}(\cdot)$ and critical path edge $\mathrm{CPE}(\cdot)$ for each vertex in the event graph. It runs in $\mathrm{O}(|V| + |E|)$ time complexity.

Remark. In our event graph, the first vertex in the topological order will always be the origin o as it is the only vertex v with $\mathrm{d}_{\mathrm{in}}(v) = 0$.

Example For event graph in figure 3.5, the time and critical path edge output from algorithm 3.2 is in table 3.1. So the vertices in the critical path for a are (o, a, b, c), for j are (o, a, b, c, h, i, j) and for w are (o, a, b, c, h, i, u, v, w).

Algorithm 3.2

Time and critical path predecessor calculation

```
input: weighted graph G, topological order O
output: time T and critical path edge CPE for each vertex in G
begin
  foreach v ∈ V(G)
    T(v) ← 0
    CPE(v) ← Undefined
  end

  foreach u ∈ O \ {o}
    foreach (u, v) ∈ δ_out(u)
      if T(v) < T(u) + w_{u,v}
        T(v) ← T(u) + w_{u,v}
        CPE(v) ← (u, v)
      end
    end
  end
end
```

Table 3.1

Time and critical path edge for vertices in event graph

vertex	T	CPE
a	1	(o, a)
b	4	(a, b)
c	8	(b, c)
g	4	(b, g)
h	8	(c, h)
i	12	(h, i)
j	14	(i, j)
u	12	(i, u)
v	16	(u, v)
w	18	(v, w)

4. Methodology

4.1 Problem description

Most of the problem description is taken from the *DISPLIB format specification* (Kloster et al., 2025a), a concise explanation is in accompanied paper (Kloster et al., 2025b).

Each instance in the *DISPLIB* library consists of set of trains and an objective to be minimized. All data is presented in JSON format (Pezoa et al., 2016). Train is defined by an operation graph which is DAG, operation has the following content:

- minimal duration
- start lower bound (optional)
- start upper bound (optional)
- list of successors
- list of required resources
  - resource name
  - release time

Data 4.1 shows an example of an operation entry in the instance JSON.

All operation graphs have one entry (start) operation, with no predecessors, and one exit (end) operation, with no successors. Each operation ends exactly when the successor operation starts (this is what we call a transition), exit operation has no end.

Resources are anything that is required by the operation, typically physical sections of the railway network. Resources can only be used by one operation at the time and are released after the end of the operation that uses it plus the defined release time.

Objective is defined by a list of objectives components, each with the following content

- train
- operation
- threshold
- coefficient (linear objective)
- increment (stepwise objective)

Remark. While the format allows both coefficient and increment to be non-zero, in practice none of the instances mix both in one objective component.

Data 4.2 shows an example of an objective component entry in the instance JSON.

The total objective delay Δ is defined as a sum of components for active operations only,

$$\Delta = \sum_{o \in O_{a,\Delta}} c_o \max\{0, t_o - \bar{t}_o\} + d_o H(t_o - \bar{t}_o), \tag{4.1}$$

where $O_{a,J}$ is a set of active operations that have an objective component, t_o is start of the operation, $\bar{t}_o$ is the threshold for the operation, c_o is the linear objective coefficient, d_o is the stepwise objective increment, and $H(\cdot)$ is Heaviside step function (Abramowitz & Stegun, 1972).

A solution to the problem is a sequence of events, each event entry corresponds to start of an operation and contains:

- train
- operation
- time

The solution is also presented in a JSON format, data 4.3 shows truncated solution example. To reiterate, the end event of an operation is the start event of its successor.

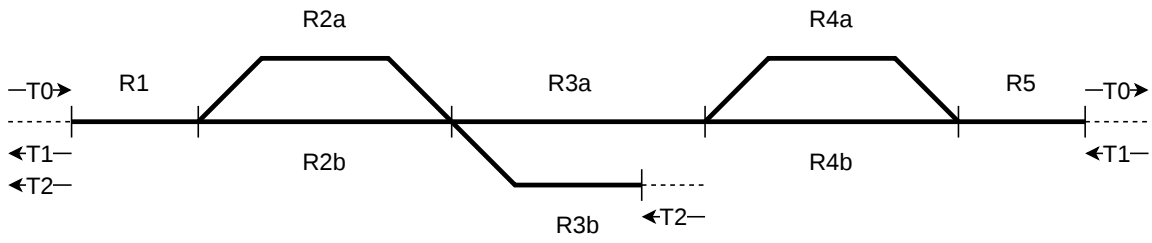
> - The sequence of events is given in chronological order, i.e., the time of each event is greater than or equal to the time of all preceding events.
> - The subsequence of operations applying to each train forms a path through that train's operations graph, starting at the entry operation and ending at the exit operation.
> - The time value of each start event satisfies the lower and upper bounds on the start time of the operation.
> - The duration from each start event to the corresponding end event must be greater than or equal to the minimum duration of the corresponding operation.
> - For any pair of operations o_1 and o_2, where:
> - the start event of o_1 precedes the start event of o_2 in the global sequence, and o_1 and o_2 belong to different trains, and
> - o_1 and o_2 use a common resource r,
>
> the following must hold:
> - the end event for o_1 precedes the start event for o_2 in the global sequence, and
> - the duration from the end event of o_1 to the start event of o_2 is greater than or equal to the release time of resource r in o_1.
>
> (Kloster et al., 2025a, Feasible solution requirements in *DISPLIB* format specification)

While the events in the solution have a time, their order in the list is still significant, as they define in which order the resources are unlocked and locked. This is to prevent a resource swap situation described in section 4.1.2.

4.1.1 Example problem

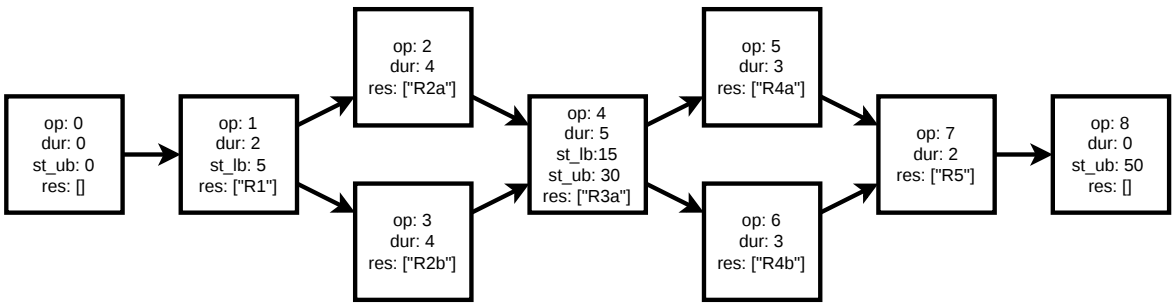
To illustrate the format we have prepared example problem, the railway network diagram is in picture 4.1. The network consists of 3 trains, 18 operations, and 8 railway sections, corresponding to resources.

Figure 4.1
Example railway network



The corresponding operation graph for train 0 (first) is in figure 4.2, JSON entry for operation 4 is in data 4.1, and JSON entry for objective component for the last operation is in data 4.2.

Figure 4.2
Operation graph for train 0



4.1.2 Resource swap

Consider the situation as in figure 4.3, where two trains a and B are passing a section of two resources X and Y in opposite direction. If we were to set events A2 and B2 to the same time, we would not have a resource collision on either resource (overlap of intervals for operations A1 and B2 on resource X or A2 and B1 on resources Y) but in reality the trains would collide.

Data 4.1

JSON operation entry

```
{
    "start_lb": 0,
    "start_ub": 50,
    "min_duration": 5,
    "resources": [
        {
            "resource": "R3a",
            "release_time": 0
        }
    ],
    "successors": [
        5,
        6
    ]
}
```

Data 4.2

JSON objective component entry

```
{
    "type": "op_delay",
    "train": 0,
    "operation": 8,
    "threshold": 25,
    "coefficient": 1
}
```

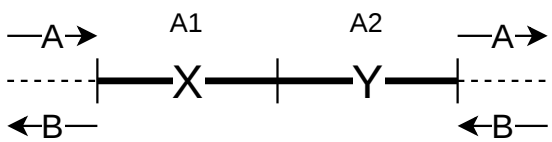
Data 4.3

JSON solution truncated

```
{
    "events": [
        ...,
        {
            "train": 0,
            "operation": 5,
            "time": 15
        },
        {
            "train": 1,
            "operation": 4,
            "time": 15
        },
        ...
    ],
    "objective_value": 25
}
```

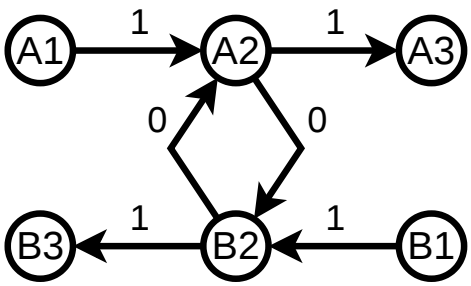
Figure 4.3

Resource swap railway



In this situation, resource Y is being acquired by A while holding X and X is being required by B while holding Y. We can detect the resource swaps using an event graph by cycle detection, like in figure 4.4 where A2 and B2 form a cycle.

Figure 4.4
Resource swap event graph



Remark. Using cycles in event graph to detect resource swaps only works if all resource precedence edges are present, otherwise we still might have a resource swap situation because the current event times do not create a collision and one or both of the edges that would create the cycle in the event graph are missing.

4.2 Problem aggregation

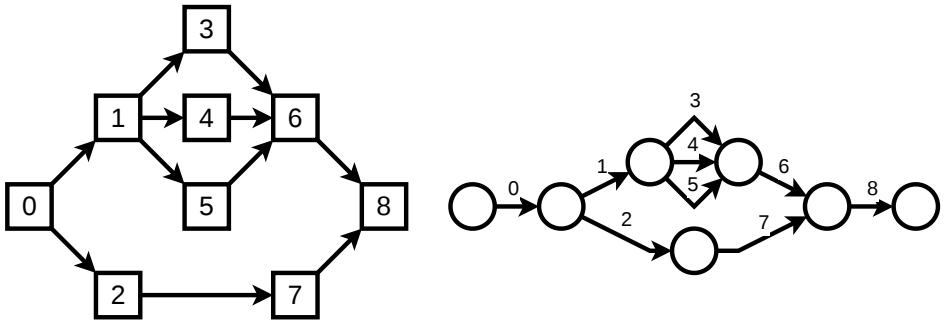
To effectively model the problem we will aggregate the operations and resources from the initial problem into several interconnected structures.

4.2.1 Junctions

First aggregation is what we call junctions, corresponding to a single event of one operation starting and one ending (except for starting and ending operations). Because we always want each operation to have single determined event for start and end, the junction graph is a directed multigraph, where each edge corresponds to an operation (as opposed to an operation graph where each operation was a vertex), figure 4.5 shows an example of an operation graph with corresponding junction graph. We use the name junctions as conceptually, the operations converge and diverge in these points as do real tracks in railway junctions.

Aside from more natural representation of using vertices as single events and edges as intervals, which translates to the events graphs, we also reduce the number of required vertices and edges, for the example in figure 4.5, it is from 9 vertices and 11 edges to 7 vertices and 9 edges; this includes the often redundant start and end events.

Figure 4.5
Operation graph and junction graph



Note: Operation graph is left, junction graph is right.

Disjoint set

To aggregate the variables into junctions, we use a *disjoint set* (Galler & Fischer, 1964). It is often called the *union-find* data structure, it is used to partition elements into non-overlapping (disjoint) sets. It has two basic operations:

- *Find:* Unary operation, returns to which set the item belongs, usually by returning the representative element.
- *Union:* Binary operation, if the two elements belong to different sets, it joins them together.

In short, each element has a parent entry, which is at the start initialized to itself and the representative element of a set always has the parent as itself. To get the representative we traverse the tree upwards via parent entries until we find the representative. When we use the union operation on two elements from different sets, the representative of one becomes a representative of the other (if they belong to the same set nothing happens).

To optimize the data structure we use path compression (flattening of the parent tree by assigning the parent of the element to its parent when traversing the tree) and union by size (always attaching smaller tree to bigger tree) (Tarjan, 1975). Both operations with optimizations are in algorithms 4.4 and 4.5.

Procedure

The procedure to create the junctions is done for each train separately.

1. Create a disjoint set for each operation of the train.
2. For each operation o we use the *union* operation on each consecutive pair of successors $\{x, y\}$, $x, y \in \mathrm{succ}(o)$.
3. Each disjoint set of operations is a set of outgoing operations $\delta_{\mathrm{out}}(j)$ for junction j.

Algorithm 4.4

Disjoint set - find

```
input: element v
output: representative s

begin
  while u ≠ parent(u)
    parent(u) ← parent(parent(u))
    u ← parent(u)
  end
  s ← u
  return s
end
```

Algorithm 4.5

Disjoint set - union

```
input: elements u and v
output: none

begin
  u ← find(u)
  v ← find(v)
  if u ≠ v
    if size(u) < size(v)
      parent(u) ← v
      size(v) ← size(v) + size(u)
    else
      parent(v) ← u
      size(u) ← size(u) + size(v)
    end
  end
end
```

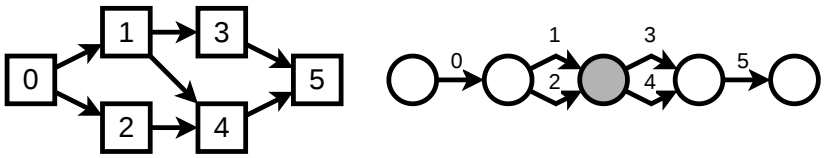
4. Add the end (sink) junction j_{end} that represents the end of the exit operation o_{exit} for the train, it has no outgoing operations.
5. Assign the source junction $\text{s}_{\text{junct}}(o) \leftarrow j$ for each operation $o \in \delta_{\text{out}}(j)$.
6. Assign the target junction $\text{t}_{\text{junct}}(o) \leftarrow \text{s}_{\text{junct}}(x)$ to any of its successors $x \in \text{succ}(o)$ (they all must have the same starting junction because of step 2.).
7. Assign the target junction $\text{t}_{\text{junct}}(o_{\text{exit}}) \leftarrow j_{\text{end}}$ to the end (sink) junction of the train.
8. Assign each junction the set of incoming operation $\delta_{\text{in}}(j) \leftarrow \{o \mid \text{t}_{\text{junct}}(o) = j\}$.

Invalid transitions

We could have a possible situation where the set of outgoing operations for a junction j is not equal to the set of successors for an incoming operation o_{in}, $\delta_{\text{out}}(j) \neq \text{succ}(o_{\text{in}})$, or vice versa the set of incoming operations for a junction j is not equal to the set of predecessors for an outgoing operation o_{out}, $\delta_{\text{in}}(j) \neq \text{pred}(o_{\text{in}})$. This generates an invalid transition $(o_{\text{in}}, o_{\text{out}})$, such that $o_{\text{in}} \in \delta_{\text{in}}(j)$ and $o_{\text{out}} \in \delta_{\text{out}}(j)$, but $o_{\text{out}} \notin \text{succ}(o_{\text{in}})$ or $o_{\text{in}} \notin \text{pred}(o_{\text{out}})$. This case is rare and only affects a few instances, but should be checked when making routing decisions.

An example of such invalid transition is in figure 4.6, where the transition (2, 3) in the third junction is invalid.

Figure 4.6
Operation graph junction graph for invalid transition



Note: Operation graph is left, junction graph is right. Junction with invalid transition (2, 3) is gray.

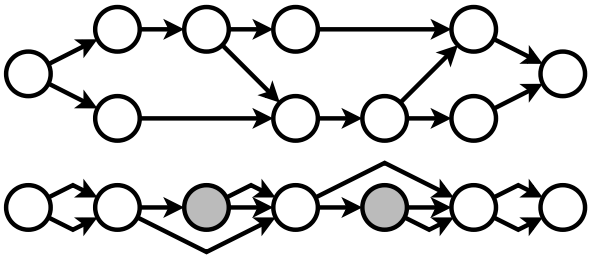
4.2.2 Levels

Second is the aggregation to levels. We group parallel junctions to levels,[1] as junctions can be parallel. This is useful to reduce the number of required vertices for each train in the event graph. An example of a junction graph and its corresponding level graph is in figure 4.7.

Each level L consists of set of junctions $j \in L$ that are parallel and thus mutually exclusive. The incoming and outgoing set of operations for each level is the union of the incoming and

[1] In (Dubach et al., 2025b) they call levels *event clusters*, the level term was taken from Venislav Varbanov.

Figure 4.7
Junction graph and its corresponding level graph



Note: Junction graph is at the top, level graph is at the bottom. Optional levels are gray.

outgoing operations of its junctions, respectively,

$$\delta_{\text{in}}(L) = \bigcup_{j \in L} \delta_{\text{in}}(j), \tag{4.2}$$

$$\delta_{\text{out}}(L) = \bigcup_{j \in L} \delta_{\text{out}}(j). \tag{4.3}$$

Each operation also has a starting (source) level and ending (target) level based on its starting (source) and ending (target) junction,

$$\mathrm{s}_{\text{level}}(o) = L \iff \mathrm{s}_{\text{junct}}(o) \in L \tag{4.4}$$

$$\mathrm{t}_{\text{level}}(o) = L \iff \mathrm{t}_{\text{junct}}(o) \in L \tag{4.5}$$

The start (source) level contains only the start (source) junction, and the end (sink) level contains only the end (sink) junction.

Required and optional levels

Not all levels need to be passed when traversing the graph. Such levels we call optional, the levels that have to be passed when traversing the graph we call required. Dubach et al. (2025b) uses the concept of required levels to generate bound constraints. We use required levels to identify border levels, which are used to create routing sections in 4.2.4. In the example from figure 4.7 the grayed out levels are optional, because they can be bypassed by an operation.

Topological generations

To aggregate the junctions into levels, we use the concept of *topological generations* (Hagberg et al., 2008). To create these generations we use adjusted Kahn's algorithm for topological

Algorithm 4.6

Topological generation algorithm

```
input: junction graph G(J, O), junctions J (vertices), operations O (edges)
output: levels (L_0, ..., L_n) (subsets of junctions)

begin
  foreach j ∈ J
    indeg(j) ← d_in(j)
  end

  k ← 0
  Z ← {J_0}

  while Z ≠ ∅
    L_k ← Z
    Z ← {}
    foreach i ∈ L_k
      foreach (i, j) in δ_out(i)
        indeg(j) ← indeg(j) − 1
        if indeg(j) = 0
          Z ← Z ∪ {j}
        end
      end
    end
    k ← k + 1
  end

  return (L_0, ..., L_n)
end
```

sorting (Kahn, 1962), where we group all junctions (vertices) currently without incoming operations (edges) into a level as shown in algorithm 4.6.

Procedure

We apply algorithm 4.6 to each train separately. We also assign each operation to each level based on the junctions and assign each operation its starting and ending level.

To get required levels, we first label all levels as required and then iterate over all operations and check which levels it bypasses (if any), and mark those as optional.

4.2.3 Routes

Now that we have aggregated distinct operation start and end events into junctions and levels, we aggregate the routing decision into routes. Each route R consists of a set of operations such that if operations x and y are in the same route and y is a successor of x then, x is the

only predecessor of y and y is the *only* successor of x, and to reduce the number of routes we force the implication the other way as well,

$$((x \in R) \wedge (y \in R) \wedge (y \in \mathrm{succ}(x))) \iff ((\mathrm{succ}(x) = \{y\}) \wedge (\mathrm{pred}(y) = \{x\})). \tag{4.6}$$

This follows the train flow network definition in (Fuchs et al., 2025). An example of how operations are grouped into routes is in figure 4.8.

Remark. If operation y is a successor of x then x is a predecessor of y,

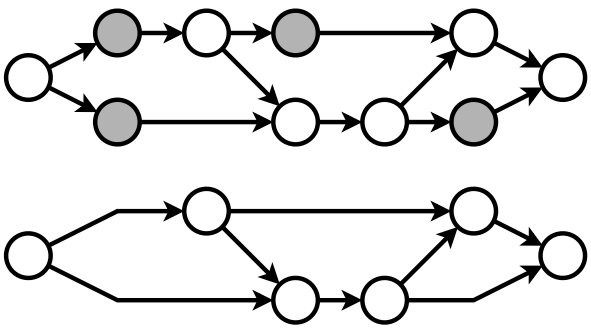
$$y \in \mathrm{succ}(x) \iff x \in \mathrm{pred}(y). \tag{4.7}$$

Activation z_R of route R is equal to activation of its operations z_o,

$$z_R = z_o, \forall o \in R. \tag{4.8}$$

Figure 4.8

Junction graph and route graph



Note: Junction graph is at the top and route graph is at the bottom. Non-routing junctions are gray.

This assignment of routes also generates a set of routing junctions (junction is routing if any route ends and/or begins there), each having a routing (flow) constraint

$$\sum_{R \in \rho_{\mathrm{in}}(j)} z_R - \sum_{R \in \rho_{\mathrm{out}}(j)} z_R = \begin{cases} 1 & \text{if } j \text{ is a start (source) junction} \\ -1 & \text{if } j \text{ is an end (sink) junction} \\ 0 & \text{otherwise} \end{cases}, \tag{4.9}$$

where z_R is the activation of R, $\rho_{\mathrm{in}}(j)$ is a set of incoming routes and $\rho_{\mathrm{in}}(j)$ is a set of outgoing routes for junction j.

For non-routing junctions, the constraint is redundant as it has only one incoming and one outgoing route which are the same.

Procedure

For each train we iterate over operations in topological order and assign them the same route if it is the only successor of its only predecessor, otherwise we create a new route. Procedure follows algorithm 4.7.

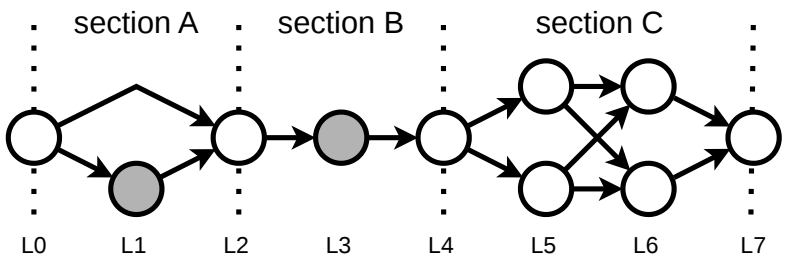
Algorithm 4.7
Route assignement

```
input: operations O = (o_0, ... o_n) in topological order
output: route assignment R : O → Z_0^+

begin
  k ← 0

  foreach o ∈ O
    if {p} = pred(o) ∧ {o} = succ(p)
      R(o) ← R(p)
    else
      R(o) ← k
      k ← k + 1
    end
  end

  return r
end
```

4.2.4 Routing sections

To separate what routing decisions are connected to each other and which are independent, we now group routes into sections. Each section starts and ends at a border level. Border levels are identified using conditions in the following subsection. Example of a junction graph being split into sections is in figure 4.9.

Figure 4.9
Junction graph with identified sections and levels



Note: L0, L2, L4, and L7 are border levels. Non-routing junctions are gray.

4.2.5 Border levels

Border level is a level that meets the following conditions:

(a) The level is required.
(b) All junctions $j \in L$ for level L are routing junctions.
(c) For each incoming operation $o_{\text{in}} \in \delta_{\text{in}}(L)$ of level L, the successor set of o_{in} is equal to the set of outgoing operations of L, $\text{succ}(o_{\text{in}}) = \delta_{\text{out}}(L)$.
(d) For each outgoing operation $o_{\text{out}} \in \delta_{\text{out}}(L)$ of level L, the predecessor set of o_{out} is equal to the set of incoming operations of L, $\text{pred}(o_{\text{out}}) = \delta_{\text{in}}(L)$.

Condition (a) ensures that each section starts and ends at a required level. Condition (b) ensures that we do not split a route by the border level. Conditions (c) and (d) ensure that any route from the level can be taken independently of which route was taken to the level and, vice versa, any route to the level can be taken independently of which route was taken from the level.

Remark. A sufficient set of conditions to replace conditions (c) and (d) is:

- The level has one junction.
- That single junction has no invalid transitions.

This is due to the nature of how we create the junctions in the first place. If a level L has a single junction j, then $\delta_{\text{in}}(L) = \delta_{\text{in}}(j)$ and $\delta_{\text{out}}(L) = \delta_{\text{out}}(j)$, and there are no invalid transitions in this junction, then the following equations hold,

$$\delta_{\text{in}}(L) = \delta_{\text{in}}(j) = \text{pred}(o),\ \forall o \in \delta_{\text{out}}(L), \tag{4.10}$$

$$\delta_{\text{out}}(L) = \delta_{\text{out}}(j) = \text{succ}(o),\ \forall o \in \delta_{\text{in}}(L). \tag{4.11}$$

Start and end levels

The start level (source) and end level (sink) are always border levels,

(a) The start level (source) and end level (sink) are always required.
(b) The start level (source) and end level (sink) have always one junction, the start (source) and end (sink) junction, respectively.
(c) There are no incoming operations for the start (source). The end (sink) has one incoming operation, the exit, which has no successor.
(d) There are no outgoing operations for the end (source). The start (source) has one outgoing operation, the entry, which has no predecessor.

Procedure

For each train we do the following:

1. Identify the border levels with the aforementioned conditions.
2. Assign consecutive sections ($S_0, \ldots, S_n$) based on border levels.
3. Add all routes R within border levels (using its operations' levels) to the specified section $S \leftarrow S \cup \{R\}$.

Example

In example from figure 4.9 we can see that L1 satisfies conditions (c), and (d) for a border level, but it is not required, and its junction is non-routing, so it does not satisfy condition (a) and (b).

We can also see that L3 satisfies conditions (a), (c), and (d), but again, its junction is non-routing, so (b) is not satisfied.

Levels L5 and L6 satisfy (a) and (b), but (c) and (d) are not satisfied. This is because the top and bottom outgoing operations of L4 have different disjunct successor sets thus different ending junctions, and, respectively, the top and bottom incoming operations of L7 have disjunct predecessor sets and thus different junctions as well.

4.2.6 Resource chunks

So far we have only done aggregation based on the operations and their successors. Now we do aggregation based on the resources. While this is inspired by Dubach et al. (2025b), the implementation may differ as the authors did not provide us with sufficient details, only the conditions that are used to create the chunks.

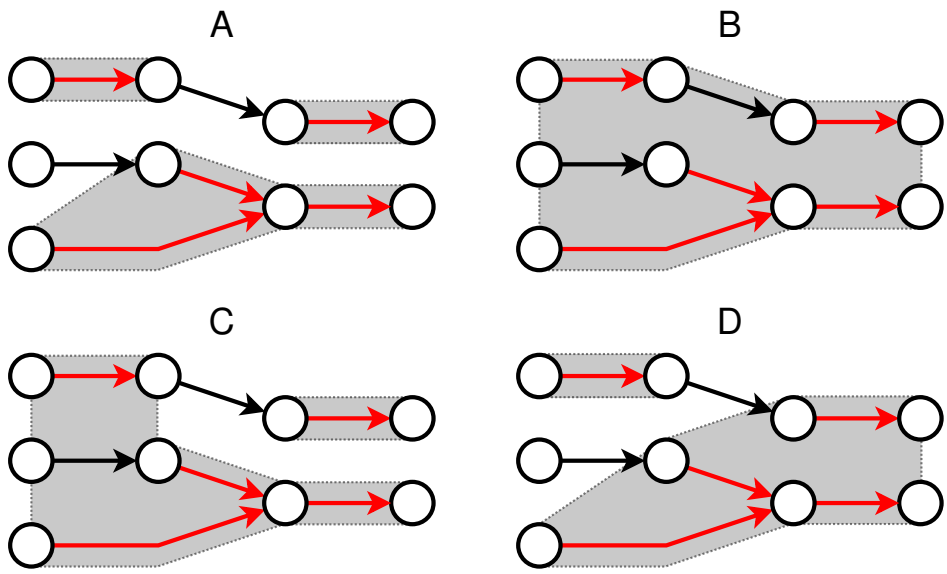
The goal is to merge the resource uses of the same resource for directly connected operations (by predecessor and successor sets) as well as merge parallel uses of the resource. What we do not want to group is reentries of resource, where on train acquires a resource, then releases it, and then acquires it again. Figure 4.10 shows an example of how operations using the same resource are merged.

The (B) case in figure 4.10 shows an invalid chunk, where the operations on the top have a reentry of the same resource, thus they cannot be in the same chunk.

While direct merges might feel sufficient, in the following section 4.3, we will be creating a data structure that grows with the amount of chunks, so reducing the number of possible chunks will be helpful to reduce the requirements of said structure.

Each chunk $c = (r_c, O_c)$ is defined by the source r_c and its set of operations O_c. We say that

Figure 4.10
Resource chunks



Note: Operations using the selected resource are red. (A) are chunks after direct merges, (B) is an invalid chunk with reentry, (C) and (D) are valid chunks after parallel merges.

the chunk is active if any of the operations $o \in O_c$ are active. Each operation o also has its set of chunk C_o, one chunk for each resource it requires.

Chunk c is active $z_c = 1$ if and only if any of the operations $o \in O_c$ are active,

$$z_c = 1 \iff \exists o \in O_c,\ z_o = 1. \tag{4.12}$$

The start of chunk is the start of its first active operation (this holds for start junction, level, and time). The end of the chunk is the end of its last active operation (this again holds for start junction, and level, end time is the end time of the last operation plus the release time for the resource).

Procedure

We apply the following procedure for each train separately and for each resource that is used by the train:

1. Get a set of all operation for a train that are using the selected resource r.
2. Create a *disjoint set* data structure like in section 4.2.1 for each operation using resource r.
3. For each operation union the sets of each predecessor and each successor with the operation of they share the same resource. These are the direct merges.

4. Iterate over the pairs of disjunct sets $\{O_a,\ O_b\}$ and check if any of the operations $y \in O_b$ are reachable from any operation in $x \in O_a$, and vice versa check if any of the operations $x \in O_a$ are reachable from any operation $y \in O_b$. If they are not reachable, union O_a and O_b. These are the parallel merges. Note: This is an iterative procedure, the following checks to other chunks are done for the whole merged set of operations, $O_c = O_a \cup O_b$.
5. Assign each chunk $c \leftarrow (r, O_c)$ its set of operations O_c and add c to the set of chunks $C_o \leftarrow C_o \cup \{c\}$ for each operation $o \in C_o$.

Remark. To check the reachability we use a simple breath first search (BFS) (Moore, 1959), from the starting set of operations, that has a cutoff based on the maximum level of the target operations to reduce the search space.

4.3 Link graph

This section describes a data structure that is used to generate implication rules for resource precedence of chunks. This will be useful to reduce the number of required variables and constrains in the following model for conflict resolution. We will also use it to explore how routing decisions change the implication rules and how these decision may cause unwanted effects for scheduling.

While the implication rules for resource precedence are discussed in (Mascis & Pacciarelli, 2002), and (D'Ariano, 2008) shows the structural implication rules for train scheduling, the only relation to routing is in (Dubach et al., 2025b) where they only use a specific set of conditions.

We will try to broaden the set of conditions that can be used to generate the structural implication rules by searching matching pairs of chunk conflicts using this data structure. Because we will be maintaining a set of links between chunks, we are calling it *link graph*.

4.3.1 Resource chunk links

Resource acquisition is done in a blocking manner. This means, that to transition from operation x to y, we need to acquire all the resources r_b, $b \in C_y$ for the chunks C_y of operation y before we can release the resources r_a, $a \in C_x$ for the chunks C_x of operation x. Because of this relationship, we say that the chunks C_x are *linked* to chunks C_y, so each chunk $a \in C_x$ has a forward link $f = (a,\ b)$ to each chunk $b \in C_y$, and each chunk $b \in C_y$ has a backward link $g = (b,\ a)$ to each chunk $a \in C_x$.

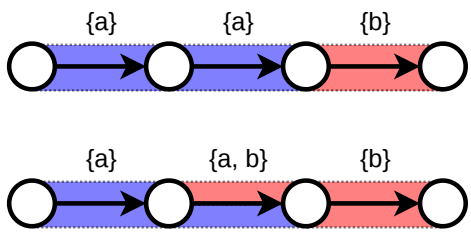
Each operation o can have multiple chunks, so we create both forward $f = (a, b)$ and backward $g = (b, a)$ for each pair of chunks (a, b), $\forall a,\ b \in C_o$, $a \neq b$, creating what we call a direct link as opposed to the aforementioned transition link. The set of all forward links will create

what we call the *forward link graph* $\mathcal{F} = (C, F)$, the set of all backward links are what we call *backward link graph* $\mathcal{G} = (C, G)$, where vertices are the chunks and edges are the links.

Remark. We keep track of forward links F_c and backward links G_c for each chunk c separately, so we can have $(c, a) \in F_c$ and also $(c, a) \in G_c$ at the same time, which is the case for direct links on chunks on the same operation o, where $c, a \in C_o$.

It is important to note that the link is active if any of the transitions (x, y) that create it are active (both x and y are active), or it is a direct link via the operation o that is active. Figure 4.11 shows both a transition and a direct link example.

Figure 4.11
Resource chunk link



Note: Operations have their chunk sets. Chunk a is blue, chunk b is red. Transition link is at the top, direct link is at the bottom.

Conflicts and precedence

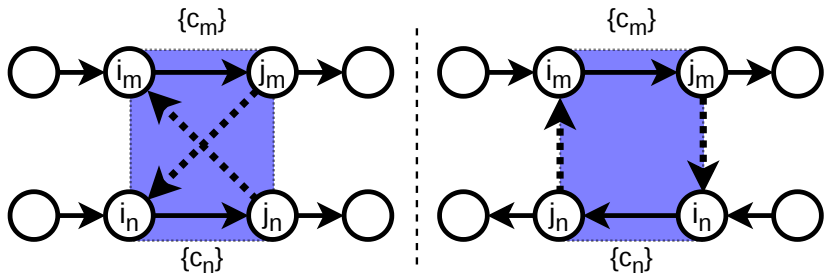
A conflict $k = \{c_m, c_n\}$ is a pair of chunks c_n and c_m on the same resource $r = r_{c_m} = r_{c_n}$ and for different trains $m = \text{train}(c_m) < \text{train}(c_n) = n$.[2] We resolve each conflict by assigning precedence $p_k = 1 - \bar{p}_k$, meaning that if $p_k = 1$, the resource r will be released by c_m before it is acquired by c_n, if $\bar{p}_k = 1$, the resource r will be released by c_n before it is acquired by c_m.

We can model this situation using alternative graph where each vertex is an event (junction or level), each edge is an operation and the alternative pair is the precedence decision. Figure 4.12 shows the alternative graph for a conflict. By assigning precedence $p_k = 1$, we select edge (j_m, i_n) from the alternative pair $((j_m, i_n), (j_n, i_m))$ and vice versa.

Remark. Without the information about the resources before and after the conflict, we cannot determine if the trains pass the resource in the same or opposite direction. This can be seen from 4.12 where both alternative graphs are equivalent.

[2] We select $n < m$ to clearly define which train is first based on the precedence.

Figure 4.12
Alternative graph for conflict



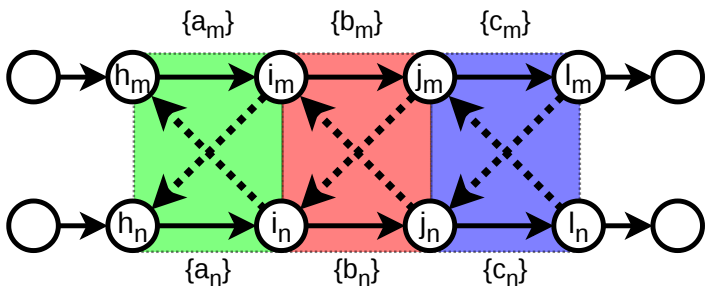
Note: Alternative edges are dotted. Chunks are in curly braces.

Conflict links

Because the trains pass the chunks continuously and in a blocking manner, there are structural implication rules for precedence for conflicts on these neighboring chunks. This is discussed in (D'Ariano, 2008), and we will paraphrase these rules here.

There are two basic cases, trains going in the same (parallel) direction and trains going in the opposite direction. The first case was already shown in section 3.3, but we will reiterate it here to show how the inferred selection in alterative graphs relates to the chunk precedence. The parallel case is shown in figure 4.13.

Figure 4.13
Conflict links - parallel case



Note: Alternative edges are dotted. Chunks are in curly braces. Conflicts are distinguished by color.

As we can see, the precedence $p_a \implies p_b$,[3] because $(i_m, h_n) \implies (j_m, i_n)$ in the alternative graph. Similarly, $p_b \implies p_a$ for $(j_m, i_n) \implies (i_m, h_n)$, so $(p_a \iff p_b) \equiv (p_a = p_b)$. Because the precedence decisions p_a and p_b imply each other, we say that conflicts k_a and k_b are linked.

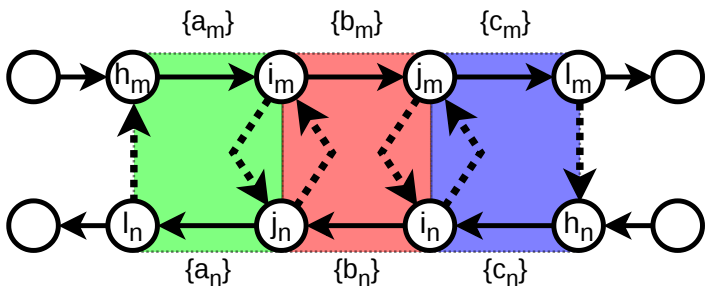
Similarly, we can also see from figure 4.13 that $p_b = p_c$, so k_b is linked to k_c. Because k_a and k_b are linked, and k_b and k_c are linked, we say that conflicts k_a, k_b and k_c form a conflict

[3]There is a slight abuse of notation where we use $p_{k_a} = p_a$.

chain $\Gamma = \{k_a,\ k_b,\ k_c\}$, whose precedence decision are equal, meaning that for all of these conflicts and their chunks, either train m or n is first, they cannot change their order.

The second case is the opposite direction case, where trains pass the resources in the opposite directions. This case is shown in figure 4.14. Using the same logic as for the parallel case, we can see that k_a and k_b are linked, $p_a = p_b$, because $(i_m, j_n) \implies (j_m,\ i_n)$ and $(j_m,\ i_n) \implies (i_m, j_n)$. Again we can also see that $p_b = p_c$, so k_a, k_b, and k_c form a chain for this case as well.

Figure 4.14
Conflict links - opposite case



Note: Alternative edges are dotted. Chunks are in curly braces. Conflicts are distinguished by color.

Finding conflict links and chains

While the shown cases are trivial, in practice, we do know which direction the trains are passing[4], so we need to use the forward and backward link graphs $\mathcal{F}$ and $\mathcal{G}$ to figure out the direction.

Remark. The main motivation for the link graph was that finding the linked pairs of conflicts exhaustively was computationally expensive, and while the inferred selection conditions in the literature are theoretically sufficient, they were impractical to check for problems of the size found in the DISPLIB.

To find if there exists a link between two conflicts $k_a = (a_m,\ a_n)$ and $k_b = (b_m,\ b_n)$ we can check the link graphs for a matching chunk links. So conflicts k_a and k_b have a parallel link if either of the following conditions hold:

(1) Chunk a_m has a forward link $(a_m,\ b_m) \in F_{a_m}$ and a_n has a forward link $(a_n,\ b_n) \in F_{a_n}$.
(2) Chunk a_m has a backward link $(a_n,\ b_m) \in G_{a_m}$ and a_n has a backward link $(a_n,\ b_n) \in G_{a_n}$

Conflicts k_a and k_b can also have an opposite link if either of the following conditions hold:

(3) Chunk a_m has a forward link $(a_m, b_m) \in F_{a_m}$ and a_n has a backward link $(a_n, b_n) \in G_{a_n}$.

[4]Trains might also be parallel in one section and opposite in another section of the track.

(4) Chunk a_m has a backward link $(a_m, b_m) \in G_{a_m}$ and a_n has a forward link $(a_n, b_n) \in F_{a_n}$

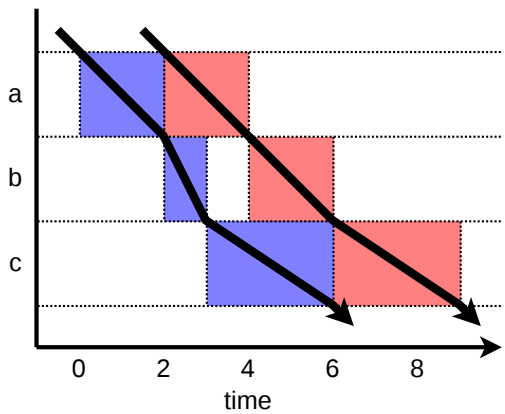
Remark. Condition (1) and (2), and (3) and (4), respectively, are symmetric. This is because if chunk a has a forward link $(a, b) \in F_a$, then chunk b always has a backward link $(b, a) \in G_b$.

Using these conditions, we can use BFS of DFS to find the entire chain of conflicts from a specific conflict, either by selecting a subset of condition for a specific direction (parallel or opposite), or any condition to find linked precedence decisions.

4.3.2 Scheduling effects of conflict chains

Because trains cannot change order within the conflict chain, they have direct effects on the possible scheduling of trains to each resource. First we will cover the parallel case. For two trains that have a parallel resource chain $\Gamma = \{k_a, k_b, k_c\}$ and on resources a, b, and c, and operations on a have a minimal duration of 2 units, on b they have 1 unit and on c they have 3 units, with the earliest possible scheduling the schedule diagram (similar to Gantt diagram) is shown in figure 4.15.

Figure 4.15
Schedule diagram - parallel chain



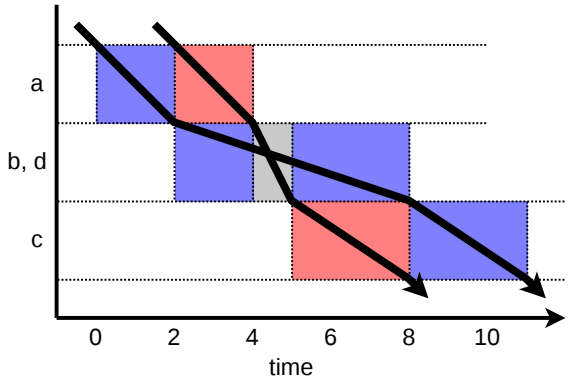
Note: Trains are distinguished by color.

If the trains had the earliest possible start for resource a at 0, then the second train would be delayed by only 3 units. If we were to require that the first train is overtaken by the second train after a and before c, we would need to break the conflict chain by removing the conflict k_b, therefore rerouting either train to not use its chunk for resource b. This overtaking situation is only good if the first train has a big gap before it is considered delayed (the threshold of the objective is high). Such schedule is shown in figure 4.16.

Because of the no swapping rule, the opposite conflict chains have a much bigger impact on the scheduling. Figure 4.17 shows the schedule diagrams for the two possible precedence cases.

Figure 4.16

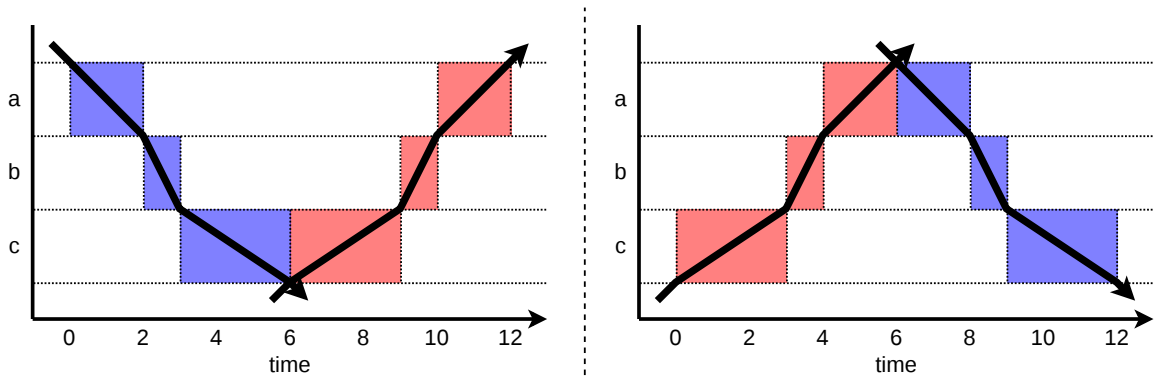
Schedule diagram - overtaking



Note: Trains are distinguished by color. Gray is overlap of intervals for resource b and d.

Figure 4.17

Schedule diagram - opposite chain



Note: Trains are distinguished by color.

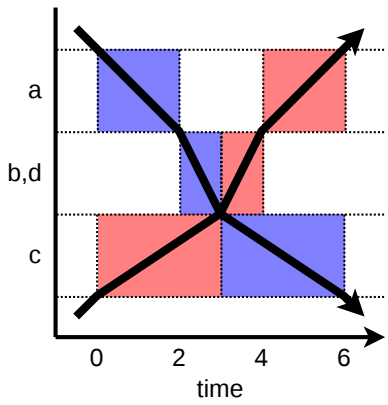
As we can see, the schedule of the second train to pass this chain has to be delayed until the first one passes through it entirely. This holds for arbitrarily long opposite chains, where the second train needs to wait until the first opposite train leaves its last chunk before the second train can enter its first chunk.

Furthermore, when looking at train routing from a safety perspective, an opposite conflict chain means that trains that are going in the opposite direction are sharing a continuous sequence of resources (usually corresponding to sections of the railway track). In a situation where the train breaks the schedule and enters the conflict chain before the other train leaves, we would have a head-on collision.

Remark. Conflict chains only impact the scheduling only if the affected trains want to pass it at the same time. If they pass the chain at different times then the precedence decision becomes trivial.

Due to these reasons, we will attempt to break up these opposite conflict chains as much as possible when making routing decisions. If we want to allow the swap of priority on resources a and c, we need to break the conflict chain by rerouting either train to not use its chunk for resource b. By doing this we allow the schedule shown in figure 4.18 where the priority is swapped. As you can see, the delay of the second train is entirely removed.

Figure 4.18
Schedule diagram - swapping



Note: Trains are distinguished by color.

Remark. While the trains in figure 4.18 have no overlapping intervals, we still need the resource d. If both trains were scheduled to b then we would not have a time collision, however they would still break the no-swapping requirement.

Implementation details

To use the link graph in practice we list all possible links activations (operations or transitions) beforehand and maintain their current activation by checking a bitmap for active operations. This is because a single link can be activated by multiple different transitions and/or direct operations, for example the bottom case in figure 4.11, forward link (a, b) has two activation counts, one direct and one transitional.

When we make a change to routes, thus changes to operations and transition, we only update the links affected by this change, as when making routing decisions we do them incrementally so only a small subset of possible operations and transitions is affected. This allows us to maintain the link graph continuously, without the overhead of recomputing it entirely for every change.

4.4 Route planning

To choose which routes will be selected, we implement a routing procedure that gradually selects routes that break the opposite conflict chains for conflicts that have overlapping chunk time intervals, or are close to each other. This routing selection is motivated by the finding in the last section, we for each opposite conflict chain, the second train can only enter the chunks in the chain only after the first train has passed them already, causing a significant delay.

4.4.1 Loss calculation

To quantify how good or bad is the current routing selection is, we calculate the loss Λ_n of each train n as the sum of opposite chain lengths $|\Gamma_k(\Omega)|$ for each corresponding train conflict $k \in K_{n,\text{close}}$, that is deemed as close,[5] based on the current route selection $\Omega = \{R_1, \ldots R_{|\Omega|}\}$,

$$\Lambda_n(\Omega) = \sum_{k \in K_{n,\text{close}}} |\Gamma_k(\Omega)|. \tag{4.13}$$

To calculate the price, loss, of the route λ_R, we have an augmented selection $\Omega_{\text{aug}} = \Omega \cup S$ where we add all routes $R \in S$ from section S and then calculate the difference in train loss by removing the route R.

$$\lambda_R = \Lambda_n(\Omega_{\text{aug}}) - \Lambda_n(\Omega_{\text{aug}} \setminus \{R\}) \tag{4.14}$$

Each route loss calculation is done only for one section S at a time, by changing the selection of routes Ω, we change the conflict chains Γ and thus the train and the route costs.

The main motivation behind this is to gauge how much does removing the route break up the opposite conflict chains for the selected conflicts.

4.4.2 Procedure

The routing procedure is done in following steps:

1. Get the average level time by randomized routing. This means that for each train we simulate passing the operation graph by selecting each successor at random with equal probability. We can get the level time by the earliest possible time for the operation by applying constraints for the lower bound for its start and minimal duration from its predecessor.

[5] How we select close conflicts is in the procedure subsection

2. Get the set of close conflicts by checking which chunks have overlapping intervals. This is done while all operations are activated. We also stretch the interval on both sides to get conflicts that are not yet overlapping, but might be when the scheduling shifts them. We assign these conflicts to trains for loss calculation.
3. Initialize current routing selection at random, similar to step 1.
4. Freeze all current routing variables z_R by fixing their bounds. This will reduce the ILP model size in preprocessing of the ILP model as the fixed variables will be substituted.
5. Select all sections that have more than one route.
6. Optimize the selected sections in random order.

The optimization of each section is done in the following steps:

1. Calculate loss λ_R for each route $R \in S$ in the section S.
2. Unfreeze variables z_R for each route in the section.
3. Minimize the total price $\sum_{R \in S} \lambda_R \; z_R$ for the section by ILP optimization.
4. Freeze all variables z_R for each route in the section.

The ILP model for optimizing a section S has the variables z_R for each route R and the flow constraints in equation 4.9, it can be written as

$$\min_{z_R} \sum_{R \in S} \lambda_R \; z_R, \tag{4.15}$$

$$\text{s.t.:} \sum_{R \in \rho_{\text{in}}(j)} z_R - \sum_{R \in \rho_{\text{out}}(j)} z_R = \begin{cases} 1 & \text{if } j \text{ is a start (source) junction} \\ -1 & \text{if } j \text{ is an end (sink) junction} \\ 0 & \text{otherwise} \end{cases}, \tag{4.16}$$

$$j \in J_S, \; z_R \in \{0, 1\}, \tag{4.17}$$

where J_S is the set of routing junctions in the section S, $\rho_{\text{in}}(j)$ is a set of incoming routes and $\rho_{\text{in}}(j)$ is a set of outgoing routes for junction j.

We iterate over sections because they are the smallest possible blocks of dependent routes and the routing decision as each section will affect the pricing of the following section, and thus optimizing all sections would be problematic as their interconnected nature makes it computationally infeasible. While a better than random order of sections could be utilized, we have not come up with a solution that would improve the loss optimization, compared to random order of sections.

While we could repeat the optimization steps or restart entirely it and try for a better solution, in practice there are no significant improvements in doing so. This is because most of the instances have some symmetry in their routing options where we might swap operations between trains without impacting the lengths of the chains.

4.5 Conflict resolution

Because 2 of the 3 winning solution in DISPLIB have used a big-M formulation (Wolsey, 1998) for the constraints implied by the routing and precedence decisions, we have decided to try a different approach based on a *path and cycle* formulation (Lamorgese & Mannino, 2019)(Lamorgese et al., 2015), which is similar to the LBBD in Dubach et al. (2025b).

This approach does not use the big-M constraints that are often problematic (Pineda & Morales, 2019)(Codato & Fischetti, 2006). Choosing the value of the M greatly influence the performance of the optimizer, values that are too small cutoff feasible solutions, leading to an incorrect model, and values that are too big weaken the LP relaxation, which then requires more nodes in the branch-and-bound search to arrive at an integer solution, and can even cause numerical instability.

Instead, we use an event graph which includes the selected alterative edges from precedence decision, and we add feasibility cuts to prevent cycles and to enforce lower-bound on delays based the current precedence variables via the critical path. This eliminates the need for big-M constrains, as the highest coefficients are the lower-bounds for the delays. The main drawback for this approach is that we may need to add an exponentially growing number of cuts to remove the infeasible solutions. This is why we use the link graph to group these precedence decisions into chains to reduce the required number of variables and cycle cuts.

4.5.1 Event graph

To generate the times of each level, we employ an event graph, where each level corresponds to one vertex of the graph (plus the origin). We add one edge for each operation from its starting level to its ending level with the weight equal to the minimum duration of the operation. We also add one edge for each conflict based on the current value of the precedence variable from the end of the first chunk to the start of the second chunk with the weight equal to its release time. Figures 4.19 and 4.20 show such event graph with both operation and precedence edges.

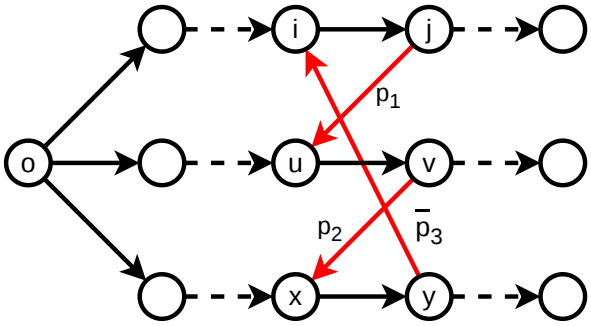
4.5.2 Cycle cuts

The MILP problem can assign values of precedence variables in such way that the selection of alternative edges is inconsistent (there exists a cycle in the graph). This situation is shown in figure 4.19, where alterative edges (j, u), (w, x) and (y, i) are part of a cycle.

Because we cannot remove the operation edges (we are working with fixed routes), we prevent the cycle by adding a no good cut on the precedence variables in the cycle Π_C,

$$\sum_{p \in \Pi_C} p \leq |\Pi_C| - 1, \tag{4.18}$$

Figure 4.19
Event graph - cycle



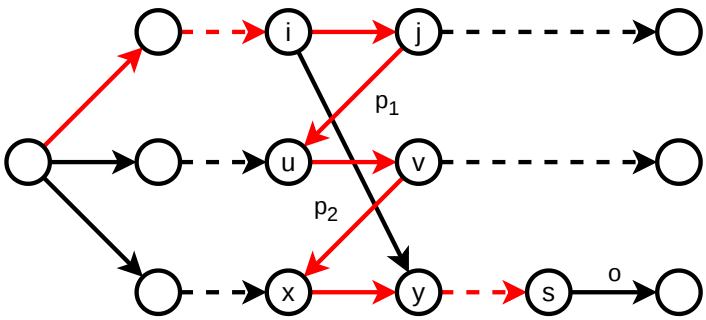
Note: Selected alterative edges are red. Alternative edges have their corresponding precedence variable.

forcing any of the variables to change their value, preventing the cycle from forming. For the example, $\Pi_C = \{p_1, p_2, \bar{p}_3\}$, $\bar{p}_3 = 1 - p_3$.

4.5.3 Path cuts

For each active operation with an objective $o \in O_{a,\Delta}$, we have delay variable Δ_o. The MILP model is trying to minimize these, but the event graph sets the earliest t_o based on its critical path. This creates a lower-bound for the Δ_o variable based on the current precedence values. Figure 4.20 show such critical path for s, generating time $\mathrm{T}(s)$.

Figure 4.20
Event graph - critical path



Note: Critical path for s is red. Alternative edges on the critical path have their corresponding precedence variable.

We can enforce the conditional lower-bound of delay Δ_o, if it is violated, by adding a critical path cut,

$$\Big(\sum_{p \in \Pi_P} p - |\Pi_P| + 1 \Big) \Big(\mathrm{T}(s) - \bar{t}_o \Big) \leq \Delta_o, \tag{4.19}$$

where Π_P is the set of precedence variables on the critical path, $\mathrm{T}(s)$ is the time of the starting level for o (in the current graph), and $\bar{t}_o$ is the objective threshold. This cut is active

only when all the precedence variable Π_P along it are the same. For the example in figure 4.20, only p_1 and p_2 are in the set $\Pi_P = \{p_1, p_2\}$, the third precedence variable is not on the critical path.

If the objective is binary, thus Δ_o is binary, indicating that $t_o > \bar{t}_o$, then we just add an implication cut,

$$\sum_{p \in \Pi_P} p - |\Pi_P| + 1 \leq \Delta_o. \tag{4.20}$$

4.5.4 Conflict discovery

We iteratively discover the conflicts by finding the first overlapping pair of chunks on the same resources. We then use the link graph to find the entire chain Γ and for the discovered conflict and use one binary precedence variable $p_\Gamma \in \{0, 1\}$ for all the conflicts in the chain, $p_\Gamma = p_k$, $\forall k \in \Gamma$.

Using the chain reduces the number of necessary variables and cycle constrains, because for a chain of 2 conflicts a and b, we would need 2 variables p_a and p_b and 2 cycle cuts $p_a - p_b \leq 0$ and $p_b - p_a \leq 0$. So for each connection in the chain, we reduce the number of necessary variables by one, and the number of necessary cycle constrains by two.

An additional advantage is that using the chain prevents resource swaps, as only possible swap without overlapping chunk intervals has to be on the same opposite conflict chain.

4.5.5 MILP model

The complete MILP model that is solved in each step is the following:

$$\min_{\Delta_o, p_\Gamma} \sum_{o \in O_{a,\Delta}} c_o \Delta_o, \tag{4.21}$$

$$\text{s.t.: all cycle cuts ,} \tag{4.22}$$

$$\text{all path cuts,} \tag{4.23}$$

where c_o is the coefficient (if continuous) or increment (if binary) of the objective for the operation o, cycle cuts have the form of equation 4.18, path cuts have the form equations 4.19 and 4.20.

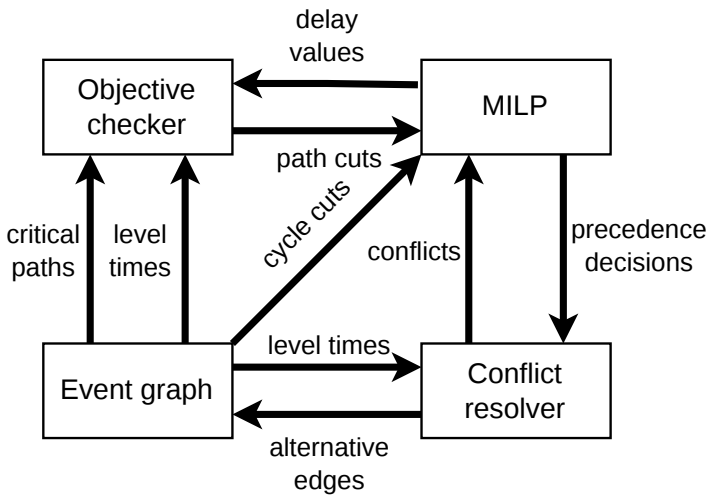
4.5.6 Procedure

The conflict resolution procedure is done in the following steps:

1. Optimize the current MILP model.
2. Update the graph with current precedence values.
3. If graph contains a cycle, add the cycle cut (eq. 4.18) for the smallest set Π_C. Go to step 1.
4. If any delays violate the conditional lower-bound, add a path cut (eq. 4.19 or 4.20) for the most violated delay. Go to step 1.
5. Check if there are any conflicts in the chunk intervals. If yes, add the whole chain for the earliest conflict. Go to step 1.
6. Stop, we have a conflict free selection of precedences.

To better illustrate how the procedure works, figure 4.21 show a diagram of what each component of the conflict resolution procedure provides for the others.

Figure 4.21
Conflict resolution diagram



While we could add multiple conflicts and/or multiple cuts per iteration, we are trying to limit the number conflicts and cuts, so the MILP model does not get cluttered and thus slows down. This is why we only add one cut or conflict per iteration and utilize the previous solution as a warm start, speeding up the computation.

Tuning the MILP optimizer

As there is a significant amount of cuts, we use a lazy constraint principle, where during the branch-and-bound procedure, each cut is treated as lazy and only added to the simplex table if it is violated. This is because by selecting a value for the precedence variable effectively disables all the cuts where the precedence variable has the opposite value, and thus they do

not need to be considered. This significantly reduces the computational demand of the model and allows us to optimize models with thousands of cuts.

We also increase the value of the mixed-integer programming gap (Gurobi Optimization, 2024), to shorten the search to solutions that might not be optimal, but are good enough. If we have spare time, we decrease the gap after each run of the conflict resolution procedure in hopes of finding a better solution.

5. Results

Software and hardware

The implementation was done in *C++* language (ISO, 2024), using *Gurobi* (Gurobi Optimization, 2024) as the MILP solver for both routing and conflict resolution. The source code is available from `https://github.com/svrc-jan/train-opt`. All computations were done on a computer with i7-9700F CPU and 16 GB of RAM.

Instance grouping

Because the DISPLIB has 112 instances, we split them into 10 groups with approximately the same size (9 to 16 instance per group) based in their initial source (encoded in the name), which also corresponds to the structure of the problem. Unless specified otherwise, the presented results will be the median across the instances inside the group, we feel that this number should be a good representation of the results as it will not be skewed by a single instance within the group and thus provide us with a valid overview that can still be distinguished between the groups.

5.1 Aggregation

To gauge how well did the aggregation of the instance data work, table 5.1 presents the initial instance data counts. We can see that the instance vary in size, with respect to number of trains, operations, resources, etc. The most significant is the difference between the number of operations compared to number of operation to resource relations (ORR, sum of the number of required resources for each operation). While the *nor* instances assign at most one resources to operation, the other instances allow multiple resources per operation.

Table 5.2 shows the aggregation counts, and the relative reductions compared to the initial data. We provide the relative improvements to see well did the aggregation procedure reduce the count required objects. For example, the operation per level ratio gives how fewer vertices we need in the event graph compared to using each operation as a vertex, i.e. how much we can parallelize the events. Similarly, the objective per route ratio gives how fewer routing decisions we need to make compared to naively having a decision (most likely a binary variable) for each operation. The most impactful is probably the ORR per chunk ratio, as grouping the related resource conflicts is going to have a direct impact on how many conflicts we need to resolve to get a conflict free solution.

[1] Operation to resource relations, will be compared to chunks

Table 5.1

Initial instance data

	nor1_crit	nor1_full	nor2_3	nor4	nor5	smi_close	smi_head	swi	wab_large	wab_small
Instance count	10	10	10	10	10	9	12	9	16	16
Trains	10	138	23	170	316	6	7	148	46	30
Objectives	10	138	23	170	316	6	7	303	46	30
Operations	464	7278	1387	17861	31156	1437	1578	9625	5748	3463
Resources	94	95	108	408	512	311	325	582	136	136
Op. transitions	625	9901	1910	26339	46270	1779	1960	9522	7424	4458
ORR[1]	445	7003	1342	17526	30524	2988	3157	35274	5672	3417

Table 5.2

Aggregation counts and relative reductions

	nor1_crit	nor1_full	nor2_3	nor4	nor5	smi_close	smi_head	swi	wab_large	wab_small
Junctions	304	4655	868	9384	16043	1198	1283	9728	4072	2467
Levels	304	4655	868	9384	16043	503	538	9417	3851	2372
Routes	397	6424	1123	16707	29744	578	578	287	4999	3015
Sections	217	3528	554	7894	13998	46	54	241	2443	1504
Res. chunks	445	7003	1342	17526	30524	785	841	10018	5108	3137
Op. / Junct.	1.53	1.56	1.61	1.93	1.95	1.20	1.20	1.00	1.41	1.40
Op. / Level	1.53	1.56	1.61	1.93	1.95	2.84	2.74	1.08	1.49	1.46
Op. / Route	1.17	1.13	1.24	1.06	1.05	2.49	2.55	20.38	1.15	1.15
ORR / Chunk	1.00	1.00	1.00	1.00	1.00	3.36	3.25	3.52	1.11	1.09

Table 5.3 show

Table 5.3
Aggregation - additional data

	smi_close	smi_headway	swi	wab_large	wab_small
Inva. tran. junct.	0.0%	0.0%	0.0%	3.0%	2.4%
Optional levels	15.7%	13.5%	0.2%	21.1%	20.7%
Direct merge	59.6%	57.3%	71.2%	8.2%	7.0%
Parallel merges	10.8%	11.7%	2.4%	1.8%	1.2%

References

Abramowitz, M., & Stegun, I. A. (1972). *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables* (Tenth Dover printing, revised, Vol. 55). Dover Publications.

Bang-Jensen, J., & Gutin, G. Z. (2008). *Digraphs: Theory, Algorithms and Applications* (2nd). Springer-Verlag.

Barbato, M., Bertoncini, A., Bianchessi, N., Tomasaz, A. B., Ceselli, A., Cordone, R., Ostuni, D., Premoli, M., Righini, G., Simone, S., & Trubian, M. (2025). Optimization and Decision Science Conference 2025: Book of Abstacts. `https://www.airoconference.it/ods2025/images/docs/ODS25_AbstractBook.pdf`

Bellman, R. (1952). On the Theory of Dynamic Programming. *Proceedings of the National Academy of Sciences of the United States of America*, *38*(8), 716–719. Retrieved April 3, 2026, from `http://www.jstor.org/stable/88493`

Boccia, M., Sasso, V. D., Lamorgese, L., Mannino, C., & Ventura, P. (2025). Optimizing Train Dispatching for the Union Pacific Railroad. `https://arxiv.org/abs/2503.19535`

Codato, G., & Fischetti, M. (2006). Combinatorial Benders' Cuts for Mixed-Integer Linear Programming. *Operations Research*, *54*(4), 756–766. `https://doi.org/10.1287/opre.1060.0286`

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th). MIT Press.

D'Amato, L., Pellegrini, P., & Trianni, V. (2025). Decentralised Multi-Agent Coordination for Real-Time Railway Traffic Management. `https://arxiv.org/abs/2502.08324`

D'Ariano, A. (January 2008). Improving Real-Time Train Dispatching: Models, Algorithms and Applications, p. 84–85.

D'Ariano, A., Pacciarelli, D., & Pranzo, M. (2007). A Branch and Bund Algorithm for Scheduling Trains in a Railway Network. *European Journal of Operational Research*, *183*(2), 643–657. `https://doi.org/https://doi.org/10.1016/j.ejor.2006.10.034`

Dijkstra, E. W. (1959). A Note on Two Problems in Connexion with Graphs. *Numerische Mathematik*, *1*(1), 269–271. `https://doi.org/10.1007/BF01386390`

Dubach, T., Fuchs, F., Lordieck, J., Corman, F., & Martin-Iradi, B. (2025a). DISPLIB competition poster. `https://www.research-collection.ethz.ch/server/api/core/bitstreams/d9c3acc3-5388-4ee2-b34a-dae5946f9224/content`

Dubach, T., Fuchs, F., Lordieck, J., Corman, F., & Martin-Iradi, B. (January 2025b). Handling Conflicts in Microscopic Railway Timetabling at Scale: A Hybrid Approach. `https://doi.org/10.2139/ssrn.5761246`

Floyd, R. W. (June 1962). Algorithm 97: Shortest Path. *Commun. ACM*, *5*(6), 345. `https://doi.org/10.1145/367766.368168`

Fuchs, F., Martin-Iradi, B., & Corman, F. (2025). Optimizing Periodic Stability in Railway Timetables: A Microscopic Model for Networks with a Macroscopic Comparison.

Journal of Rail Transport Planning & Management, *36*, 100555. https://doi.org/10.1016/j.jrtpm.2025.100555

Galler, B. A., & Fischer, M. J. (1964). An Improved Equivalence Algorithm. *Communications of the ACM*, *7*(5), 301–303. https://doi.org/10.1145/364099.364331

Gross, J. L., Yellen, J., & Zhang, P. (2013). *Handbook of Graph Theory* (2nd). CRC Press.

Gurobi Optimization, L. (2024). Gurobi Optimizer Reference Manual. https://www.gurobi.com

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring Network Structure, Dynamics, and Function Using NetworkX. In G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th python in science conference (scipy 2008)* (pp. 11–15).

Hert, D., Baca, T., Petracek, P., Kratky, V., Penicka, R., Spurny, V., Petrlik, M., Vrba, M., Zaitlik, D., Stoudek, P., Walter, V., Stepan, P., Horyna, J., Pritzl, V., Sramek, M., Ahmad, A., Silano, G., Bonilla Licea, D., Stibinger, P., … Saska, M. (July 2023). MRS Drone: A Modular Platform for Real-World Deployment of Aerial Multi-Robot Systems. *Journal of Intelligent & Robotic Systems*, *108*, 1–34. https://doi.org/10.1007/s10846-023-01879-2

Homer, S., & Selman, A. L. (2011). *Computability and Complexity Theory*. Springer Science & Business Media.

Chai, S., Yin, J., D'Ariano, A., Liu, R., Yang, L., & Tang, T. (March 2024). A Branch-and-Cut Algorithm for Scheduling Train Platoons in Urban Rail Networks. *Transportation Research Part B: Methodological*, *181*, 102891. https://doi.org/10.1016/j.trb.2024.102891

ISO. (2024). *ISO/IEC 14882:2024: Programming Language C++*. International Organization for Standardization. Geneva, Switzerland. https://iso.org

Kahn, A. B. (November 1962). Topological Sorting of Large Networks. *Communications of the ACM*, *5*(11), 558–562. https://doi.org/10.1145/368996.369025

Kloster, O., Luteberget, B., Mannino, C., & Sartor, G. (2025a). DISPLIB Format Specification [Accessed 2026/05/04]. https://displib.github.io/displib_spec_2025-09-17.pdf

Kloster, O., Luteberget, B., Mannino, C., & Sartor, G. (2025b). DISPLIB: A Library of Train Dispatching Problems. https://arxiv.org/abs/2509.12254

Lamorgese, L., & Mannino, C. (2019). A Non-compact Formulation for Job-Shop Scheduling Problems in Traffic Management. *Operations Research*, *67*(6), 1586–1609. https://doi.org/10.1287/opre.2018.1824

Lamorgese, L., Mannino, C., & Piacentini, M. (2015). Optimal Train Dispatching by Benders'-like Reformulation. *Transportation Research Part C: Emerging Technologies*, *60*, 42–57.

Leutwiler, F., & Corman, F. (February 2022). A Logic-Based Benders Decomposition for Microscopic Railway Timetable Planning. *European Journal of Operational Research*, *303*. https://doi.org/10.1016/j.ejor.2022.02.043

Light, B. (2024). The Principle of Optimality in Dynamic Programming: A Pedagogical Note. https://arxiv.org/abs/2302.08467

Mascis, A., & Pacciarelli, D. (2002). Job-Shop Scheduling with Blocking and No-Wait Constraints. *European Journal of Operational Research*, *143*(3), 498–517. https://doi.org/https://doi.org/10.1016/S0377-2217(01)00338-1

Moore, E. F. (1959). The Shortest Path Through a Maze. *Proc. Internat. Sympos. Switching Theory 1957, Part II*, 285–292.

Pezoa, F., Reutter, J. L., Suarez, F., Ugarte, M., & Vrgoč, D. (2016). Foundations of JSON Schema. *Proceedings of the 25th International Conference on World Wide Web*, 263–273.

Pineda, S., & Morales, J. M. (2019). Solving Linear Bilevel Problems Using Big-Ms: Not All That Glitters Is Gold. *IEEE Transactions on Power Systems*, *34*(3), 2464–2467. https://doi.org/10.1109/TPWRS.2018.2889357

Roy, B., & Sussmann, B. (1964). *Les Problèmes d'Ordonnancement avec Contraintes Disjonctives* (Note D.S. No. 9 bis). Direction Scientifique SEMA. Paris, France.

Strotmann, C. (2007). *Railway Scheduling Problems and Their Decomposition* [Doctoral dissertation, University of Osnabrück] [Dissertation im Fachbereich Mathematik / Informatik]. https://uni-osnabrueck.de

Tarjan, R. E. (1975). Efficiency of a Good But Not Linear Set Union Algorithm. *Journal of the ACM*, *22*(2), 215–225. https://doi.org/10.1145/321879.321884

Wolsey, L. A. (1998). *Integer programming*. Wiley.